

Desafio Técnico – Parte 2

(Interface de Gerenciamento do Monitor de Servidores)

Contexto

Na primeira etapa, você desenvolveu o motor em Python do nosso Monitor de Disponibilidade de Servidores, operado via linha de comando e configurado através de arquivos JSON estáticos.

Embora o backend seja eficiente, executar o sistema diretamente editando arquivos de texto não é escalável nem seguro para os usuários no dia a dia. Para transformar essa ferramenta em uma solução completa, precisamos de uma interface web moderna, intuitiva e segura, permitindo que os administradores gerenciem as configurações, os servidores e os usuários notificados sem precisar tocar no terminal.

O Desafio

Seu objetivo é desenvolver uma aplicação SPA (Single Page Application) utilizando **Angular (versão 17 ou superior)** chamada **Painel de Controle do Monitor**.

Essa aplicação será a interface visual responsável por ler, validar e editar os três arquivos de configuração JSON definidos na etapa anterior: **monitorConfig.json**, **userInfo.json** e **servers_config.json**.

Ajuste de Nomenclatura e Padronização de Arquivos

Como parte da evolução do projeto para um padrão de produção mais maduro e consistente, sua primeira tarefa nesta etapa será refatorar a nomenclatura dos arquivos de dados herdados da Parte 1. Para garantir a uniformidade (utilizando snake_case e respeitando as regras de plural e singular de acordo com a estrutura dos dados), os arquivos deverão ser renomeados e organizados localmente da seguinte forma:

- 1) **monitorConfig.json** => **monitor_config.json** (Objeto de configuração global do sistema e SMTP);
- 2) **userInfo.json** => **users_info.json** (Coleção de dados dos administradores cadastrados);
- 3) **servers_config.json** => **servers_pool.json** (O repositório dos servidores disponíveis para monitoramento).

Requisitos Técnicos (Angular)

1. Estrutura e Navegação (Routing):

A aplicação deve ser dividida de forma clara em três telas principais (ou abas bem definidas):

Configurações do Monitor (monitor_config.json): Formulário para editar os dados do servidor SMTP (host, porta, credenciais) e etc.

Gerenciamento de Usuários (users_info.json): Uma visão de listagem (tabela ou cards) que permita adicionar, editar e remover os administradores cadastrados para receber os e-mails.

Catálogo de Servidores (servers_pool.json): Uma visão que permita o CRUD completo (Criação, Leitura, Atualização e Deleção) dos servidores que podem ser monitorados.

2. Formulários e Validações Reativas (Reactive Forms):

A interface deve garantir que o operador não insira dados corrompidos que quebrem o monitor em Python. Implemente validações estritas nos formulários:

E-mails: Validar o formato correto do e-mail do usuário no cadastro de administradores.

Endereços IP: Validar se o IP inserido no cadastro de servidores segue o padrão IPv4 válido (0.0.0.0 a 255.255.255.255).

Portas: Validar se a porta do servidor e SMTP são números inteiros válidos dentro do intervalo de portas de rede.

Campos Obrigatórios: Impedir o salvamento caso existam campos vazios (como Hostname ou Username).

3. Experiência do Usuário (UX/UI) e Componentização:

O candidato está livre para utilizar bibliotecas de componentes de mercado (como Angular Material, Tailwind CSS ou PrimeNG) para garantir um visual limpo e profissional.

Feedback Visual: A tela deve exibir mensagens claras de sucesso ao salvar as alterações ou mensagens de erro detalhadas caso as validações falhem.

Componentes Reutilizáveis: Avaliaremos a capacidade de criar componentes limpos (ex: botões, inputs customizados ou tabelas de listagem reutilizáveis).

4. Diferenciais Operacionais (Opcional, mas bem avaliado):

Exportação dos Arquivos: Um botão que permita baixar os arquivos JSON atualizados diretamente pelo navegador, prontos para serem colocados na pasta do script Python.

Simulador de monitor_list.txt: Uma tela simples onde o operador possa marcar (com checkboxes) os servidores cadastrados manualmente e atualizar o arquivo monitor_list.txt contendo uma indicação dos hostnames selecionados para próximas verificações.

Requisitos Estruturais: Ambiente 100% Local e Isolado

Para fins de avaliação, deve ser possível executar a aplicação de forma totalmente local na máquina do avaliador, sem qualquer dependência de servidores externos, bancos de dados em nuvem ou APIs remotas.

A persistência e manipulação dos dados dos arquivos JSON devem seguir uma das duas abordagens abaixo (à sua escolha):

Arquitetura Baseada em Serviços (Em Memória): Os dados dos JSONs são carregados inicialmente no estado da aplicação e manipulados em memória durante o uso da tela.

Mock API Local: Utilização de uma ferramenta leve de mock que rode localmente em ambiente de desenvolvimento (como o json-server via npm) para ler e salvar as alterações diretamente em arquivos locais na sua máquina de desenvolvimento.

Apresentação:

Após a entrega desse Desafio será marcada uma reunião de apresentação, onde você fará uma demonstração do sistema completo:

1. Mostrando o fluxo para adicionar um novo servidor e validar um IP inválido.
2. Explicando a arquitetura de módulos/componentes escolhida e como foi estruturado o gerenciamento de dados no Angular.

Instruções de Entrega:

1. **Código-Fonte:** O projeto Angular deve ser entregue no mesmo repositório da Parte 1 (em uma pasta separada, ex: /frontend). Por exemplo, se na Parte 1 foi entregue em um arquivo compactado, a Parte 2 deve seguir o mesmo padrão de entrega.

2. **README.md (atualizado):** Inclua as instruções de como instalar as dependências do ecossistema Angular (`npm install`) e o comando para rodar o servidor de desenvolvimento local (`ng serve`).